

Stargate: Security Analysis and Mission-Aware Firewall for the CubeSat Space Protocol

Owen Ramsey 
New Mexico State University
owenr@nmsu.edu

Carlos Gonzalez-Cortes 
Universidad de Santiago de Chile
carlos.gonzalez.c@usach.cl

Efrén López-Morales 
New Mexico State University
elopezm@nmsu.edu

Abstract—The CubeSat Space Protocol (CSP) is widely used for communication in CubeSat missions, yet its security model and attack surface remain largely unexplored. In this work, we present a systematic security analysis of CSP and identify fundamental limitations in its trust assumptions, coarse-grained network abstractions, privileged management services, application command semantics, and lack of mission-context awareness. Based on these findings, we develop a taxonomy of CSP attacks spanning four classes: network identity abuse, management service abuse, application semantic abuse, and mission context abuse. We then introduce Stargate, a modular, mission-aware firewall that enforces CSP-specific policies derived from this taxonomy, including node and service authorization, command-level filtering, and mission-context constraints. We implement Stargate and evaluate it on a virtual testbed based on a real-world CubeSat mission and flight software. Stargate successfully blocks all 11 representative attacks across the four attack classes while introducing only 1.42 ms of additional median round-trip latency. Our results demonstrate that protocol- and mission-aware policy enforcement provides a practical approach for securing CSP-based spacecraft networks.

Index Terms—satellite security, cubesat space protocol (CSP), spacecraft cybersecurity, firewalls, network security

I. INTRODUCTION

Modern CubeSats increasingly rely on reusable networking stacks and modular flight software for spacecraft development. Among these, the CubeSat Space Protocol (CSP) has emerged as one of the most adopted communication protocols for CubeSat missions, providing lightweight routing, and management services for communication between onboard subsystems and ground stations. CSP has been incorporated into open-source flight software [1], [2], commercial spacecraft platforms [3], [4], and operational mission control systems [5], [6], making it an important component of the small satellite ecosystem.

Historically, satellite systems relied on security by obscurity, e.g., limited public documentation and proprietary implementations [7]. Today, however, the advent of commercial off-the-shelf (COTS) spacecraft hardware, open-source flight software, and accessible ground station infrastructure has substantially lowered the barrier to understanding satellite communication systems [8]. Consequently, the security of widely deployed communication protocols such as CSP has become increasingly important.

Prior work has shown that CSP implementations contain vulnerabilities [9], [10] and has introduced CPS-focused defenses such as Intrusion Detection Systems (IDS) [11].

However, despite CSP’s widespread adoption, its security has not been systematically studied. Specifically, the protocol’s own security model, attack surface, and inherent security limitations remain largely unexplored.

To address this gap, we *perform the first systematic security analysis of CSP*. Specifically, we characterize the protocol’s threat model and attack surface, providing a systematic assessment of the protocol from a security perspective.

Our analysis demonstrates that CSP’s security challenges extend beyond software vulnerabilities and instead arise from fundamental limitations in the protocol’s design. In particular, CSP provides only coarse-grained communication abstractions based on node identifiers and service ports, while the semantics of spacecraft telecommands are encoded within application-layer payloads. Furthermore, the protocol assumes trusted participants and provides no native mechanism for enforcing mission-specific security policies. Consequently, many security-critical operations cannot be authorized using CSP’s native abstractions alone.

For example, the Space Attack Research and Tactic Analysis (SPARTA) framework, which provides a knowledge base of adversarial techniques and countermeasures for spacecraft systems, includes the *Secure Command Modes* countermeasure (CM0055), which recommends constraining spacecraft commands based on conditions such as command source and geographic location [12]. However, CSP does not provide mechanisms for enforcing such command policies.

Based on our analysis’ findings, we developed a *taxonomy of CSP attacks* that organizes protocol threats into four classes: network identity abuse, management service abuse, application semantic abuse, and mission context abuse. Rather than classifying individual exploits, the taxonomy captures the fundamental protocol abstractions that attackers can abuse and serves as the foundation for the design of our defense.

Motivated by our analysis’ results, we propose *Stargate*, a mission-aware firewall for CSP networks. Unlike traditional network firewalls, which enforce policies using IP addresses, and transport-layer ports, Stargate is designed to understand and secure CSP-specific communications.

We designed Stargate to meet both functional and non-functional requirements. The functional requirements are based on the proposed attack taxonomy and define the security policies needed to mitigate each attack class. The non-functional requirements address deployment concerns, in-

cluding modularity and mission configurability, enabling the firewall to be adapted across diverse CSP-based missions.

To evaluate Stargate, we developed a virtual testbed based on a real-world CubeSat mission and implemented 11 representative CSP attack scenarios using a real-world flight software.

Our results show that Stargate successfully mitigates all 11 attack scenarios across all four attack categories in our taxonomy while introducing only 1.42 ms of additional median packet latency, demonstrating that mission-aware CSP security can be achieved with limited runtime overhead.

In summary, this paper makes the following **contributions**:

- **Security Analysis:** We present, to the best of our knowledge, the first systematic security analysis of the CubeSat Space Protocol (CSP), characterizing its threat model, attack surface, and fundamental security limitations.
- **Attack Taxonomy:** We develop the first taxonomy of CSP attacks, organizing threats according to the protocol abstractions they abuse rather than individual exploits.
- **CSP Firewall:** We design and implement Stargate, a modular, mission-aware firewall whose functional requirements are based on the proposed taxonomy, enabling policy enforcement based on CSP identities, management services, application semantics, and mission context.
- **Virtual Testbed Evaluation:** We evaluate Stargate on a virtual testbed based on a real-world CubeSat mission and representative attacks implemented on real flight software and measure the latency overhead Stargate introduces.

In the spirit of open and reproducible science, this paper's source code and experimental results are available online¹.

II. BACKGROUND

In this section we lay out key concepts including the CubeSat Space Protocol (CSP) (Sec. II-A), the CSP Management Protocol (CMP) (Sec. II-B), CSP adoption (Sec. II-C), and a Representative CSP Mission Network (Sec. II-D).

A. The CubeSat Space Protocol

The CubeSat Space Protocol (CSP) is a lightweight protocol designed for resource-constrained satellite systems [13]. CSP provides routing, transport, management services, and interface drivers, forming a complete communication stack for both ground and space segment communications.

CSP follows a packet-based design with a compact header format that encodes source and destination addresses, ports, and control flags. The main CSP implementation, *libcsp* [13], uses a header optimized for low overhead and efficient processing on embedded hardware. This design makes CSP well-suited for microcontrollers and real-time operating systems commonly used in space systems.

Two major versions of CSP are supported by *libcsp* [13]. We focus our implementation and evaluation on CSP 1, which is used by our target flight software and testbed. While CSP 2 introduces changes to the protocol header, the security

limitations examined in this work largely arise from CSP's communication and authorization model and are therefore not specific to the CSP 1 header format.

CSP Communication Model. CSP follows a service-oriented communication model in which each spacecraft subsystem is assigned a unique *node identifier (ID)* and exposes its functionality through one or more *ports* [14]. Applications, such as telemetry, and network management, communicate by exchanging CSP packets addressed to a destination node and port. Upon receiving a packet, a *CSP router* first determines whether the packet should be forwarded or delivered locally based on the destination node ID. If the packet is destined for the local node, it is dispatched to the associated port, otherwise it is forwarded to a default route or interface.

CSP Routing. CSP employs a distributed routing architecture in which every CSP-enabled node can function as both a communication endpoint and a router. Each node maintains its own routing table and forwards packets toward their destination. This decentralized design improves flexibility but provides no centralized enforcement point where communication policies can be applied [14].

CSP Interfaces. CSP is transport-independent and can operate over multiple communication interfaces. Common interfaces include Controller Area Network (CAN), Inter-Integrated Circuit (I²C), serial links, and ZeroMQ (ZMQ) [15].

CSP Security Features. CSP provides a limited set of security mechanisms in its commonly used implementations, mainly XTEA and HMAC-SHA1 for encryption and authentication. However, authentication, integrity, and confidentiality are not enforced at the protocol level, and are instead left to higher-layer mechanisms or mission-specific extensions. Furthermore, XTEA and HMAC-SHA1 are fundamentally weak by modern cryptographic standards [13].

Recent efforts toward CSP version 2 aim to introduce optional security features. However, these enhancements are not yet widely deployed in operational systems.

B. CSP Management Protocol (CMP)

CSP uses service ports to identify functionality exposed by a node. Of the 64 possible CSP ports, ports 0–7 are reserved for standardized CSP services, while the remaining ports are available for mission-specific applications [14]. These standardized services provide functionality for network management, diagnostics, and spacecraft operation, including node identification, interface statistics, memory access, clock synchronization, routing configuration, and system reboot [16].

Although these services support legitimate spacecraft operations, several expose security-sensitive functionality when accessed by an unauthorized node. For example, PEEK and POKE provide memory access, ROUTE_SET modifies routing information, CLOCK modifies system time, and the reboot service can restart a CSP node.

C. CSP Adoption in Operational Missions

The CubeSat Space Protocol is widely adopted and has been integrated into multiple open-source and commercial flight

¹<https://github.com/cps-sec/stargate-csp-firewall>

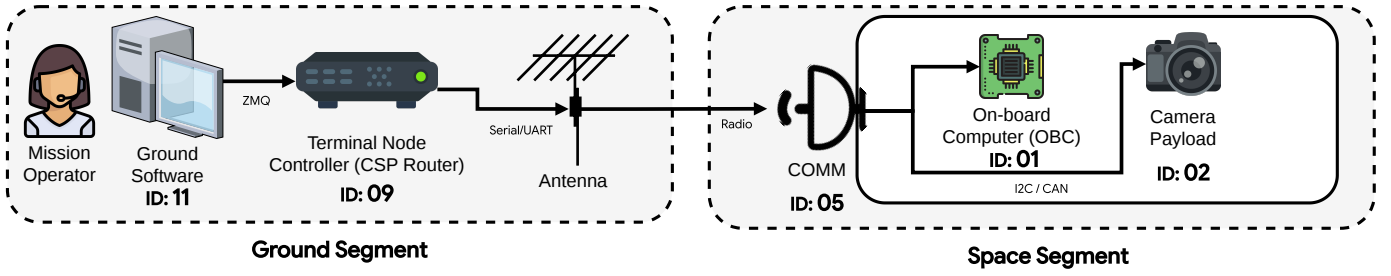


Fig. 1. CSP network architecture of the real-world SUCHAI CubeSat mission [2].

software stacks, including the SUCHAI Flight Software [2], and is supported by organizations such as the German Space Operations Center [6]. Importantly, GomSpace, a spacecraft manufacturer has adopted CSP as their de facto standard for most of their hardware [17], [3].

To better understand the extent of CSP adoption, we analyzed publicly available satellite information from SatNOGS [18] and the nanosats database [19]. Our analysis identified 104 operational satellites that use CSP for communications in some capacity. For example, several missions use GomSpace’s NanoCom AX.100 transceiver which implements CSP communications by default [16]. Our CSP-enabled satellite missions dataset is available online².

D. Representative CSP Mission Network

To ground our research in a realistic deployment, we consider the network architecture of the SUCHAI operational CubeSat mission that employs CSP [2].

Fig. 1 depicts the CSP mission network. The architecture is divided into a *ground segment* and a *space segment*, which communicate through a radio link. The ground segment consists of the mission operator, the ground software, a Terminal Node Controller (TNC), and the ground antenna. The ground software generates CSP packets that are forwarded to the TNC, which interfaces with the radio hardware and transmits the packets over the wireless link. The TNC also acts as a CSP router using a predefined CSP routing table.

The space segment consists of the spacecraft’s On-Board Computer (OBC) and one or more onboard subsystems, such as a camera payload. The OBC executes the CSP stack and serves as the primary communication gateway for the spacecraft. Upon receiving a CSP packet from the radio subsystem, the OBC routes the packet to the destination subsystem based on the destination node identifier and service port. Each subsystem is assigned a unique CSP node identifier and exposes its functionality through one or more CSP services.

III. CUBESAT SPACE PROTOCOL SECURITY ANALYSIS

To systematically identify the security challenges introduced by the CubeSat Space Protocol (CSP), we first analyze its communication model, management services, and operational

assumptions from a security perspective. Specifically, we (i) define the CSP threat model (Sec. III-A), (ii) the protocol attack surface (Sec. III-B), and (iii) develop a taxonomy of representative CSP attacks based on the identified attack vectors and trust boundaries (Sec. III-C).

A. Threat Model

Attacker capabilities. We consider a CSP-based satellite mission in which Stargate is deployed as a policy-enforcement point on the authorized communication path between the ground segment and spacecraft. We assume that CSP traffic reaching the spacecraft through this path traverses Stargate. The adversary can generate arbitrary CSP packets through the authorized ground communication infrastructure, for example through compromised ground software, operator misuse, an insider, or a secondary ground system connected to the mission network. The attacker possesses knowledge of valid CSP node identifiers, service ports, and publicly documented management services, enabling the construction of syntactically correct CSP packets targeting spacecraft subsystems.

Attacker objectives. The attacker’s objective is to invoke unauthorized spacecraft functionality, disrupt mission operations, or access sensitive information by abusing legitimate CSP services. Representative attacks include unauthorized invocation of CSP Management Protocol (CMP) services, misuse of application-specific telecommands, and denial-of-service through excessive command transmission.

Out of scope. We assume that CSP traffic using the protected ground-space communication path traverses Stargate. Adversaries capable of bypassing this enforcement point, for example by directly transmitting to the spacecraft over an independent RF link, compromising communication infrastructure downstream of Stargate, or compromising an onboard CSP node, are outside the scope of this work. Attacks targeting the physical layer, radio-frequency jamming, cryptographic mechanisms, key management, or vulnerabilities in specific CSP implementations are also outside the scope of this work.

B. CSP Attack Surface

We focus on the CSP-specific attack surface, in particular, we go beyond TCP/IP network connectivity and includes network identifiers, ports, management services, application-layer commands, and mission context.

²<https://github.com/cps-sec/csp-satellites-list>

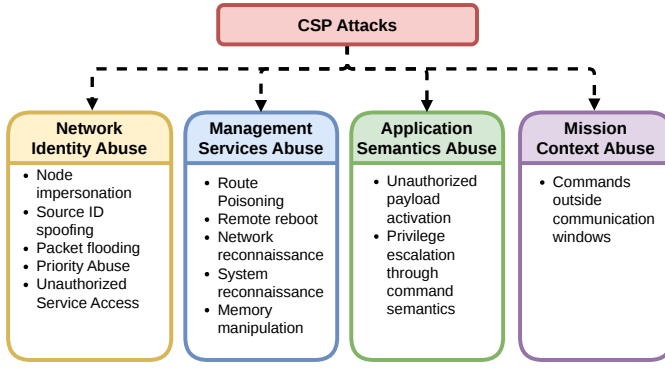


Fig. 2. CSP Attack Taxonomy

Node Identifiers. Every CSP-enabled subsystem is assigned a unique node ID. An attacker capable of transmitting arbitrary CSP packets may target any reachable node, including critical spacecraft components such as the Command and Data Handling (C&DH), or mission payloads.

Service Ports. Each node exposes functionality through one or more CSP ports. While ports provide logical separation between services, they represent only coarse-grained access control. Once a packet is delivered to a service, CSP does not distinguish between authorized and unauthorized operations.

Management Services. As discussed in Sec. II-B, CSP includes the CSP Management Protocol (CMP), which provides standardized services such as routing. These services are intended for spacecraft operation but also expose functionality that could be abused. For example, CMP routing features can be abused for route poisoning.

Application Command Semantics. Mission functionality is typically implemented through application-specific telecommands whose semantics are encoded within the packet payload rather than the CSP header. For example, two packets may have the same source ID, destination ID, and service port, yet one requests a temperature reading while the other initiates a firmware update. Since CSP only exposes the node identifiers and service ports, it cannot distinguish between these fundamentally different operations, preventing authorization decisions based solely on CSP headers.

Mission Context. The legitimacy of many spacecraft commands depends on the current operational context of the mission. Commands that are valid during one mission phase may be unsafe or unauthorized during another. Examples include payload activation outside imaging windows, or maintenance operations. These constraints are external to the CSP protocol and cannot be enforced by native CSP mechanisms.

C. CSP Attack Taxonomy

Based on the attack surface described in the previous section, we derive a taxonomy of attacks against CSP-based spacecraft networks depicted in Fig. 2. Rather than classifying attacks according to specific exploits, we categorize them according to the protocol abstraction they abuse.

Network Identity Abuse. The first class of attacks exploits CSP network identifiers, including source and destination node IDs, service ports, and packet priorities. Since CSP assumes that participating nodes are trusted, an attacker capable of injecting CSP packets may perform *node impersonation* by spoofing the source ID of a legitimate node, *unauthorized node communication* by sending packets to spacecraft subsystems that should not be reachable, *service enumeration* by probing ports to discover exposed applications, *unauthorized service access* by invoking privileged CSP or application services, *priority abuse* by marking malicious traffic as high priority to compete with legitimate communications, or *traffic flooding* by overwhelming critical nodes with excessive packets. These attacks exploit the lack of protocol-level authentication and authorization rather than implementation vulnerabilities. In particular, because CSP node identifiers are not cryptographically authenticated, a source ID alone cannot establish the identity of the sender.

Management Service Abuse. The second class targets the CSP Management Protocol (CMP). Representative attacks include *information disclosure* by reading sensitive memory with PEEK, *memory manipulation* by modifying runtime state with POKE, *route poisoning* by altering packet forwarding paths through ROUTE_SET, *time manipulation* by changing the spacecraft clock to disrupt mission scheduling, and *remote reboot* by forcing an unexpected system restart. These attacks exploit privileged management functionality intended for trusted operators.

Application Semantic Abuse. The third class targets application-level command semantics. Many spacecraft applications expose multiple telecommands through a single CSP service port, with the requested operation encoded within the application payload rather than the CSP header. Representative attacks include *firmware update abuse* by invoking software update commands through an otherwise authorized service, *unauthorized subsystem control* by issuing reset, shutdown, or mode transition commands, and *payload activation* by enabling mission payloads without operator authorization. These attacks exploit CSP’s inability to distinguish between different application commands based solely on node identifiers and service ports.

Mission Context Abuse. The final class targets mission-specific operational context. Unlike semantic attacks, the command itself may be legitimate, but its execution violates mission constraints. Representative attacks include *out-of-window command execution* by transmitting telecommands outside authorized ground contact periods, *mission-phase violations* by issuing deployment or calibration commands during nominal operations, and *state-dependent command abuse* by activating payloads or changing operating modes while the spacecraft is in a protected state such as safe mode. Since these policies depend on the spacecraft’s operational state rather than the CSP protocol, they cannot be enforced using native CSP mechanisms alone.

D. Representative Attack Scenarios

The following attack scenarios illustrate how each category of the proposed taxonomy may be exploited in practice.

Rogue CSP Node (Network Identity Abuse). A rogue CSP node joins the network and attempts to communicate with the spacecraft without authorization. By targeting valid spacecraft node IDs and service ports, the attacker can issue unauthorized requests, probe exposed services, or flood critical subsystems with malicious traffic. Since CSP assumes that participating nodes are trusted, native CSP mechanisms cannot distinguish legitimate operators from unauthorized network participants.

Route Poisoning (Management Service Abuse). An attacker abuses the CMP `ROUTE_SET` service to modify the spacecraft's routing table. By redirecting traffic to a rogue node, creating invalid routes, or blackholing legitimate communication paths, the attacker can intercept, misroute, or disrupt CSP communications between spacecraft subsystems and the ground segment. Since `ROUTE_SET` is intended for trusted network reconfiguration, unauthorized access to this service enables direct manipulation of the spacecraft's communication topology.

Unauthorized Command on an Authorized Port (Application Semantic Abuse). An attacker sends a privileged telecommand through an otherwise authorized CSP service port. For example, the same application port may accept both a benign telemetry request and a firmware update command. Although the CSP headers are identical, the requested operations have vastly different security implications. Since the command is encoded within the application payload rather than the CSP header, CSP cannot distinguish between these operations using node identifiers and service ports alone.

Command Outside an Authorized Communication Window (Mission Context Abuse). An attacker transmits an otherwise legitimate telecommand outside an authorized ground contact window. Although the command is valid, it violates mission-specific communication policies and should not be accepted. Since communication windows are external to the CSP protocol, native CSP mechanisms cannot enforce these temporal constraints.

E. Security Analysis Summary

Our security analysis demonstrates that securing CSP networks requires more than traditional packet filtering based on network addresses and ports. CSP security risks arise at multiple layers, including network identities, privileged management services, application command semantics, and mission-specific operational constraints. Consequently, effective protection requires a policy enforcement mechanism capable of authorizing *who* may communicate, *which* protocol services may be accessed, *what* application commands may be executed, and *when* those commands are permitted according to mission context. These observations directly motivate the design of Stargate, a CSP mission policy-aware firewall which we now describe.

IV. STARGATE FIREWALL DESIGN

Traditional network firewalls are designed for Internet Protocol (IP) networks, where security policies are enforced using packet attributes such as source and destination IP addresses, transport-layer ports, and protocol identifiers. These abstractions are effective for conventional enterprise networks because application services are exposed through well-defined TCP or UDP ports. In contrast, spacecraft systems using the CubeSat Space Protocol (CSP) employ a different communication model based on node identifiers, CSP service ports, management services, and mission-specific telecommands.

Furthermore, CSP traffic is commonly transported over communication technologies such as ZeroMQ tunnels, CAN buses, serial links, or radio frames. In these deployments, a conventional firewall only observes the underlying transport protocol and lacks visibility into CSP headers and application payloads. Consequently, it cannot enforce CSP-specific security policies, including restrictions on node identities, management services, application commands, or mission-dependent operational constraints.

To address these limitations, we propose Stargate, a policy-aware firewall specifically designed for CSP networks. Rather than relying on traditional IP semantics, Stargate inspects CSP packets and evaluates them against mission-defined policies derived from the security analysis presented in Sec. III. The following subsections describe the design requirements and architecture of the proposed firewall.

A. Design Requirements

Based on the security analysis presented in the previous section, we follow the below requirements.

1) *Functional Requirements:* The following four functional requirements are derived directly from the CSP attack taxonomy (Sec. III-C) and define the core security capabilities that Stargate must provide.

FR-1: Identity-aware Policy Enforcement The firewall shall enforce communication policies based on CSP identity to restrict unauthorized network communication.

FR-2: Service-aware Policy Enforcement The firewall shall selectively authorize privileged CSP management services to prevent management services abuse attacks.

FR-3: Semantic-aware Policy Enforcement The firewall shall inspect application payloads to distinguish and authorize individual telecommands exposed through the same CSP service port to prevent application semantics abuse attacks.

FR-4: Mission-aware Policy Enforcement The firewall shall enforce mission-specific operational policies, such as communication windows and mission phases, when authorizing telecommands to prevent mission context abuse attacks.

2) *Non-Functional Requirements:* The following non-functional requirements allow Stargate to be integrated into diverse CSP-based missions.

NR-1: Modular Architecture Unlike traditional network firewalls, CSP deployments vary significantly across missions in terms of communication interfaces, network topology, onboard applications, and supported telecommands.

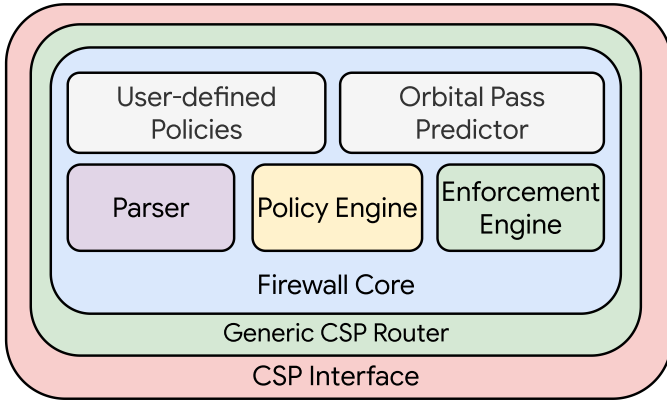


Fig. 3. Stargate Firewall Modular Architecture

Therefore, Stargate has a modular architecture that allows mission-specific components, including CSP interfaces, protocol parsers, and policy modules, to be added, replaced, or extended without modifying the entire firewall.

NR-2: Mission Configurability Stargate is designed to support mission-specific security policies, e.g., authorized node IDs, and other operational constraints.

B. Stargate Firewall Architecture

Figure 3 illustrates the architecture of Stargate. The firewall is deployed at the ground-space communication boundary, where it intercepts all CSP traffic before it reaches the spacecraft. Incoming packets first enter the *CSP Interface*, which provides an abstraction over the underlying communication medium. This modular interface satisfies NR-1 by allowing Stargate to be deployed across diverse CSP-based missions without modifying the core firewall logic.

Packets are then forwarded to the *Generic CSP Router*, which receives and forwards CSP packets through the firewall processing pipeline. The *Parser* extracts security-relevant information from each packet, including source and destination node identifiers, service ports, packet priorities, CMP operations, and application-specific telecommands. These attributes provide the input required for policy evaluation.

The extracted information is evaluated by the *Policy Engine*, which implements the functional requirements derived from the CSP security analysis (Sec. III). The policy engine evaluates packets against multiple classes of security policies, including network identity policies (FR-1), management service policies (FR-2), application semantic policies (FR-3), and mission context policies (FR-4). Mission-specific policies are loaded from the *User-defined Policies* file (NR-2), while mission timing constraints are supplied by the *Orbital Pass Predictor*, enabling enforcement of communication windows.

Finally, the policy decision is passed to the *Enforcement Engine*, which determines whether the packet should be forwarded, dropped, or logged.

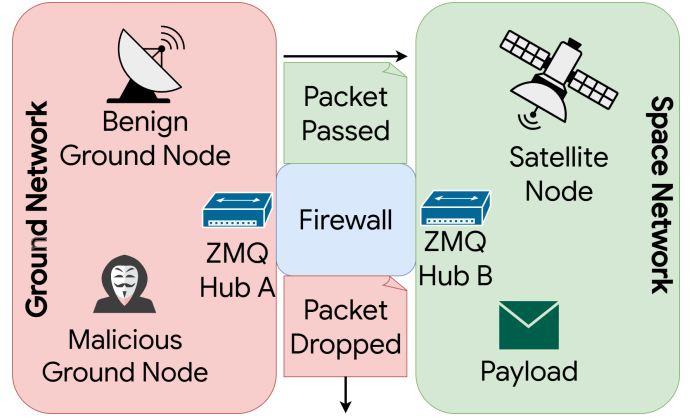


Fig. 4. CSP Virtual Testbed

C. Policy Engine Architecture

The Policy Engine depicted in Fig. 5 is responsible for evaluating parsed CSP packets against mission-defined security policies. Unlike traditional firewalls that base decisions on network-layer attributes, Stargate performs multi-dimensional policy evaluation using the information extracted by the parser. Specifically, the Policy Engine evaluates each packet against four classes of policies derived from the CSP threat taxonomy and functional requirements. A packet is forwarded only if it satisfies all applicable policies; otherwise, it is rejected and logged.

Network Identity Policies (FR-1). Network identity policies regulate communication based on CSP source and destination node identifiers, service ports, and packet priorities. These policies ensure that only authorized nodes are permitted to communicate with protected spacecraft services and prevent unauthorized or rogue CSP nodes from accessing mission-critical resources.

Management Service Policies (FR-2). Management service policies control access to privileged CSP Management Protocol (CMP) services. Operations such as PEEK, POKE, ROUTE_SET, CLOCK, and REBOOT can be selectively authorized or denied according to mission requirements, preventing unauthorized administrative operations.

Application Semantic Policies (FR-3). Application semantic policies provide fine-grained authorization of spacecraft telecommands. Since multiple telecommands may share the same CSP service port, the parser extracts the application command from the packet payload before policy evaluation. This enables Stargate to distinguish between benign operations and privileged commands that are otherwise indistinguishable at the CSP protocol layer.

Mission Context Policies (FR-4). Mission context policies evaluate operational constraints that are external to the CSP protocol. Examples include communication windows, mission phases, and spacecraft operational modes. Consequently, a command that satisfies all network and application-level

policies may still be denied if it violates mission-specific operational constraints.

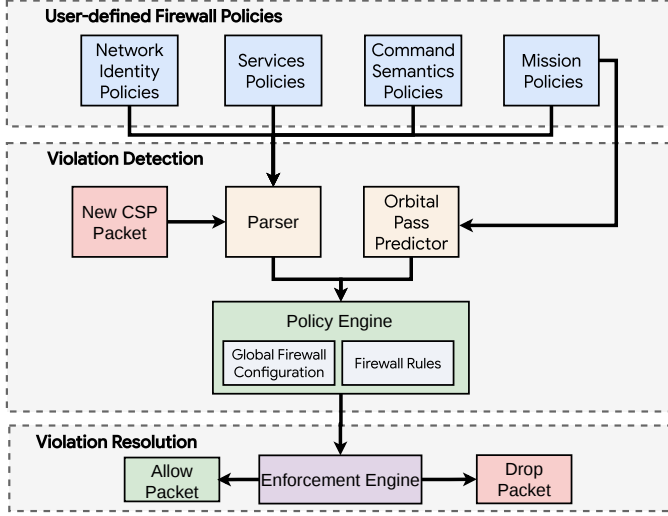


Fig. 5. Policy Engine Diagram. Inspired by [20]

V. IMPLEMENTATION

We implemented Stargate in C using the libcsf library [13]. Security policies are specified in a YAML configuration file.

Mission context policies are implemented using the Skyfield library [21], which computes satellite visibility from Two-Line Element (TLE) data. During runtime, the orbital pass predictor determines whether the spacecraft is within an authorized communication window and supplies this information to the policy engine. We use publicly available TLE data obtained from CelesTrak [22].

A. CSP Virtual Testbed

Based on the architecture of the real-world CubeSat mission discussed in Sec. II-D, we developed the virtual CSP testbed shown in Fig. 4. The testbed includes five Docker containers that emulate the main components of a CSP-enabled mission. All nodes use **libcsf**. The **benign ground node** represents an authorized ground station that generates legitimate CSP commands, while the **malicious ground node** represents an adversary capable of transmitting crafted CSP packets toward the spacecraft. Both ground nodes connect to **ZMQ Hub A**, which represents the ground-side CSP network.

The **firewall** is positioned between the ground and space networks so that all CSP traffic destined for the spacecraft must traverse it. Packets permitted by the configured security policies are forwarded to **ZMQ Hub B**, while packets that violate a policy are dropped before entering the space network. ZMQ Hub B connects the firewall to the spacecraft-side CSP nodes. The **satellite node** runs *real open-source spacecraft flight software* with CSP support, providing a realistic target for evaluating the firewall rather than a synthetic CSP application. The **payload** represents an additional onboard CSP subsystem communicating with the satellite node.

VI. EVALUATION

We evaluate Stargate by answering the following research questions:

- **RQ1:** Can Stargate mitigate representative attacks from each class of the CSP attack taxonomy? (Sec. VI-B)
- **RQ2:** What runtime overhead does Stargate introduce to CSP communication? (Sec. VI-C)

A. Experimental Setup

We evaluate Stargate using the virtual CSP testbed depicted in Fig. 4 and discussed in Sec. V-A.

Experiments were conducted in a VirtualBox virtual machine running Ubuntu 25.04 with 8 virtual CPUs and 16 GB of RAM. For each attack scenario, we first executed the attack without policy enforcement to confirm its effect on the target CSP node. We then repeated the attack with the corresponding Stargate policy enabled and recorded whether the malicious packet was forwarded or dropped.

For mission-context experiments, we used Skyfield and publicly available TLE data from CelesTrak to determine communication windows between the spacecraft and a ground station. The resulting communication state was provided to Stargate’s policy engine to determine whether telecommands were authorized.

B. Attack Mitigation Effectiveness

To answer **RQ1** we evaluated whether Stargate can mitigate representative attacks from each of the four classes identified by our CSP security analysis. Table I summarizes the evaluated attacks, their effects, the policy mechanism used for mitigation, and the observed result.

1) *Network Identity Abuse:* We evaluated four attacks targeting CSP network-level attributes: rogue-node communication, unauthorized service access, priority abuse, and packet flooding. For the rogue-node experiment, the legitimate ground station (node 10) was permitted to communicate with the spacecraft, while traffic originating from the malicious ground station (node 11) was denied. Stargate correctly forwarded traffic from the authorized node and dropped traffic originating from the unauthorized node.

We similarly verified service-port and priority enforcement by transmitting packets containing both authorized and restricted values. Packets targeting restricted ports or using prohibited priorities were dropped, while permitted packets were forwarded. Finally, the rate-limiting policy restricted command transmission to five commands per minute, preventing the malicious node from exceeding the mission-defined rate.

2) *Management Service Abuse:* We evaluated five security-sensitive CSP management operations: `PEEK`, `POKE`, `ROUTE_SET`, `CLOCK`, and `REBOOT`. Without policy enforcement, the malicious ground node was able to invoke these operations against the target CSP node, enabling memory disclosure, memory modification, routing-table modification, system-time manipulation, and remote reboot.

With Stargate enabled, all five operations were blocked using service- and payload-level policies. CMP operations

sharing the same service port were distinguished using their command semantics, demonstrating that port-level filtering alone is insufficient for protecting CSP management.

3) *Application Semantic Abuse*: To evaluate semantic-aware enforcement, we transmitted both an authorized and a restricted telecommand through the same CSP service port. Because the packets share the same CSP source, destination, and service port, a policy operating exclusively on CSP header fields cannot distinguish between the two operations.

Stargate parsed the application payload and identified the requested telecommand. The authorized command was forwarded, while the restricted command was dropped. This experiment demonstrates that application-semantic inspection enables command-level authorization beyond CSP’s native node and port abstractions.

4) *Mission Context Abuse*: We evaluated mission-aware enforcement by transmitting the same otherwise authorized telecommand both inside and outside an authorized communication window. Communication windows were determined using the orbital-pass predictor described in Sec. IV.

When the spacecraft was within an authorized communication window, Stargate forwarded the command after it satisfied the remaining policies. When the same command was transmitted outside the communication window, Stargate dropped the packet. Thus, identical CSP traffic can receive different authorization decisions based on mission context that is not represented within the CSP packet itself.

Our evaluation focuses on communication-window policies as one representative form of mission context; mission-phase and spacecraft-state enforcement are not evaluated in the current prototype.

C. Overhead

To answer **RQ2** we evaluate the overhead introduced by Stargate, we measured the round-trip time (RTT) of CSP ping messages with and without the firewall enabled. A total of 100 RTT measurements were collected per experiment and repeated five times. Table II reports the average minimum, maximum, mean, median, and mode across all experiments.

The results indicate that Stargate introduces a median RTT increase of 1.42 ms and a mean increase of 11.60 ms. The difference between the median and mean, together with the observed maximum RTTs, indicates substantial variability in the virtualized measurement environment. Therefore, these measurements should be interpreted as an initial characterization of end-to-end communication overhead rather than an isolated measurement of firewall processing cost. The reported RTT measurements include the overhead introduced by virtualization and containerization, which may increase latency and jitter compared to a native deployment [23].

Because Stargate is deployed on the ground segment, its computational requirements are less constrained than those of an onboard deployment.

D. Evaluation Summary

Our evaluation demonstrates that Stargate successfully mitigates representative attacks across all four classes of our CSP

attack taxonomy, including network identity abuse, management service abuse, application semantic abuse, and mission context abuse. These results show that combining CSP-aware packet inspection with mission-specific policy enforcement enables finer-grained authorization than CSP’s native node and port abstractions alone. Stargate achieves this protection while introducing a median RTT overhead of only 1.42 ms. Overall, the results demonstrate that Stargate can enforce mission-aware security policies with modest communication overhead.

VII. RELATED WORK

Specialized Firewalls. FLOWGUARD [20] introduced a policy-aware firewall for Software-Defined Networks (SDNs) that detects and resolves firewall policy violations caused by dynamic OpenFlow rule updates and changing network state. The framework performs network-wide flow analysis and automatically enforces access control policies as network configurations evolve. In contrast, Stargate targets the CubeSat Space Protocol (CSP), enforcing security policies based on CSP network identities, management services, application command semantics, and mission context.

CubeSat Space Protocol Security. Driouch *et al.* [11] proposed a distributed intrusion detection system (IDS) for CubeSat networks that performs anomaly detection on CSP-over-CAN communications using deep learning. Their architecture distributes intrusion detection by executing lightweight inference onboard the spacecraft while performing computationally intensive model training at the ground segment. The proposed IDS detects attacks such as denial-of-service, fuzzing, and replay under CubeSat resource constraints. However, the approach operates on CAN traffic characteristics, including packet timing, CAN identifiers, and payload data, making it dependent on the underlying CAN transport rather than the semantics of the CubeSat Space Protocol itself. Consequently, it does not analyze CSP services, application commands, or mission-specific communication policies. In contrast, Stargate performs protocol-aware policy enforcement by inspecting CSP packet fields and application semantics, enabling mission-defined access control based on CSP identities, management services, application command semantics, and mission context.

Havermans *et al.* [24] investigated the security of space communication protocol implementations through grammar-based and coverage-guided fuzzing. Their study evaluated open-source implementations of multiple space protocols, including the official `libcsp` implementation, identifying implementation vulnerabilities such as memory safety errors and input validation flaws that could be remotely exploited. While this work improves the robustness of protocol implementations by discovering software vulnerabilities, it does not address protocol-level security policies or runtime access control. In contrast, Stargate focuses on enforcing mission-defined security policies over CSP communications, preventing unauthorized or unsafe protocol interactions even when the protocol implementation itself is free of software vulnerabilities.

TABLE I
EFFECTIVENESS OF STARGATE AGAINST REPRESENTATIVE CSP ATTACKS ACROSS THE FOUR ATTACK CLASSES IN OUR TAXONOMY.

Attack Class	Attack	Effect Without Stargate	Enforcement Policy	Result
Network Identity Abuse	Rogue node	Unauthorized node sends spacecraft commands	Node identity	Blocked
	Unauthorized service access	Accesses a restricted CSP service	Service port	Blocked
	Priority abuse	Sends traffic using a restricted priority	Packet priority	Blocked
	Packet flooding	Exceeds the permitted command rate	Rate limiting	Blocked
Management Service Abuse	Memory disclosure (PEEK)	Reads memory from the target node	CMP semantics	Blocked
	Memory modification (POKE)	Writes attacker-controlled data to memory	CMP semantics	Blocked
	Route poisoning (ROUTE_SET)	Modifies CSP routing information	CMP semantics	Blocked
	Time manipulation (CLOCK)	Modifies the spacecraft system clock	CMP semantics	Blocked
	Remote reboot (REBOOT)	Forces the target CSP node to restart	Service port	Blocked
Application Semantic Abuse	Unauthorized telecommand	Executes a restricted command on an authorized port	Command semantics	Blocked
Mission Context Abuse	Out-of-window command	Executes a valid command outside an authorized contact window	Mission context	Blocked

TABLE II
STARGATE ROUND-TRIP TIME (RTT) OVERHEAD

Metric	No Firewall (ms)	Firewall (ms)	Δ (ms)	Change
Min	3.00	3.50	+0.50	+16.67%
Max	603.33	900.00	+296.67	+49.17%
Mean	51.76	63.36	+11.60	+22.41%
Median	10.83	12.25	+1.42	+13.11%
Mode	6.67	8.67	+2.00	+29.99%

VIII. DISCUSSION

A. Limitations

CSP Versions. Our analysis focuses on CSP version 1, which remains widely used in existing CSP deployments and flight software. Although CSP version 2 introduces changes to the packet header and addressing scheme, the fundamental security concerns identified in this work also apply to CSP v2. Therefore, the main conclusions of our security analysis and the policy principles underlying Stargate extend to CSP v2.

Security Analysis Scope. Our security analysis primarily examines CSP’s identity, trust, service exposure, application semantics, and mission-context assumptions. We do not evaluate the security of cryptographic implementations or conduct cryptographic attacks against CSP. Consequently, vulnerabilities arising from the implementation or configuration of authentication, encryption, or key-management mechanisms are outside the scope of this work.

B. Future Work

CSP Authentication and Authorization. A second direction is the development of authentication and authorization mechanisms for CSP. Rather than relying solely on network-level identity and port-based access control, such mechanisms could provide authenticated identities and fine-grained authorization for CSP services and application commands. We plan to investigate an authentication and authorization protocol that can operate on top of CSP while remaining suitable for the resource constraints of CubeSat systems.

IX. CONCLUSION

We presented a systematic security analysis of the CubeSat Space Protocol (CSP), identifying security limitations across network identities, management services, application semantics, and mission context. Based on these findings, we developed a taxonomy of CSP attacks and designed Stargate, a modular, mission-aware firewall that enforces policies across these four dimensions. We evaluated Stargate using a virtual testbed based on a real-world CubeSat mission, where it successfully blocked all representative attacks while introducing only 1.42 ms of additional median round-trip latency. Our results demonstrate that protocol- and mission-aware policy enforcement provides a practical approach for improving the security of CSP-based spacecraft networks.

ACKNOWLEDGMENT

This work was supported in part by the Computing Research Association’s UR2PhD program and by startup funds from New Mexico State University.

REFERENCES

- [1] C. E. Gonzalez, C. J. Rojas, A. Bergel, and M. A. Diaz, "An architecture-tracking approach to evaluate a modular and extensible flight software for cubesat nanosatellites," *IEEE Access*, vol. 7, pp. 126 409–126 429, 2019.
- [2] Space and Planetary Exploration Laboratory (SPEL), "Suchai flight software v2," <https://gitlab.com/spel-uchile/flight-software/suchai-f-light-software-v2>, 2026, gitLab Repository, accessed July 20, 2026.
- [3] J. D. C. CHRISTIANSEN, "Cubesat space protocol (csp)-network-layer delivery protocol for cubesats and embedded systems," Tech. Rep. GS-CSP-1.1, GOMSpace ApS, Tech. Rep., 2011.
- [4] U. Planta, J. Rederlechner, M. Strohmeier, M. Fischer, and A. Abbasi, "Satbleed: Security of commoditized communication modules in satellites," in *2026 IEEE Symposium on Security and Privacy (SP)*, 2026, pp. 2904–2921.
- [5] European Space Agency, "GOMX-5 (GomSpace Express-5) - eoPortal," <https://www.eoportal.org/satellite-missions/gomx-5>, n.d., accessed: 2026-07-20.
- [6] German Aerospace Center (DLR). (2024, Mar.) German space operations center commands small satellite with next-generation software via cubesat space protocol. DLR Blog. Accessed: 2026-08-04. [Online]. Available: <https://www.dlr.de/en/blog/archive/2024/german-space-operations-center-commands-small-satellite-with-next-generation-software-via-cubesat-space-protocol>
- [7] J. Willbold, M. Schloegel, M. Vögele, M. Gerhardt, T. Holz, and A. Abbasi, "Space odyssey: An experimental software security analysis of satellites," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1–19.
- [8] E. López-Morales, U. Planta, G. Marra, C. González, J. Hopkins, M. Garoosi, E. Obreque, C. Rubio-Medrano, and A. Abbasi, "Honeysat: A network-based satellite honeypot framework," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2026.
- [9] National Institute of Standards and Technology, "CVE-2016-8596 Detail," 2016, national Vulnerability Database (NVD). Accessed: 2026-07-15. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2016-8596>
- [10] —, "CVE-2016-8597 Detail," 2016, national Vulnerability Database (NVD). Accessed: 2026-07-15. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2016-8597>
- [11] O. Driouch, S. Bah, and Z. Guennoun, "Distributed intrusion detection system for cubesats, based on deep learning packets classification model," in *2024 Security for Space Systems (3S)*. IEEE, 2024, pp. 1–8.
- [12] The Aerospace Corporation, "SPARTA: Secure Command Modes (CM0055)," SPARTA – Space Attack Research and Tactic Analysis, accessed: 2026-08-10. [Online]. Available: <https://sparta.aerospace.org/countermeasures/CM0055>
- [13] LibCSP Developers, "The cubesat space protocol — cubesat space protocol," <https://libcsp.github.io/libcsp/>, 2026, libCSP Documentation, accessed July 20, 2026.
- [14] —, "Network topology," <https://libcsp.github.io/libcsp/topology.html>, 2026, libCSP Documentation, accessed July 13, 2026.
- [15] —, "The protocol stack," <https://libcsp.github.io/libcsp/protocolstack.html>, 2026, libCSP Documentation, accessed July 13, 2026.
- [16] GomSpace, *NanoGround User Manual*, GomSpace, 2025, mAN-1072373-1, Version 1.4. [Online]. Available: <https://gomspace.com/wp-content/uploads/2025/09/NanoGround-User-Manual-MAN-1072373-1-1.4-1.pdf>
- [17] —, "Nanomind a3200 on-board computer," <https://gomspace.com/hop/subsystems/on-board-computers/nanomind-a3200.aspx>, 2023, accessed: 2025-09-19.
- [18] Libre Space Foundation. (2026) Satnogs: The global open network of satellite ground stations. Accessed: 2026-08-04. [Online]. Available: <https://satnogs.org/>
- [19] Gunter's Space Page. (2026) Nanosats database. Database of nanosatellite and CubeSat missions. Accessed: 2026-08-04. [Online]. Available: <https://www.nanosats.eu/database>
- [20] H. Hu, W. Han, G.-J. Ahn, and Z. Zhao, "FLOWGUARD: Building robust firewalls for software-defined networks," in *Proceedings of the Third Workshop on Hot Topics in Software Defined Networking (HotSDN '14)*. New York, NY, USA: Association for Computing Machinery, August 2014, pp. 97–102. [Online]. Available: <https://doi.org/10.1145/2620728.2620749>
- [21] B. Rhodes, "Skyfield: Elegant astronomy for python," <https://rhodesmii.org/skyfield/>, 2026, accessed July 20, 2026.
- [22] T. Kelso, "CeresTrak," <https://celestrak.org/>, 2026, accessed July 20, 2026. [Online]. Available: <https://celestrak.org/>
- [23] K. Suo, Y. Zhao, W. Chen, and J. Rao, "An analysis and empirical study of container networks," in *IEEE INFOCOM 2018-IEEE Conference On Computer Communications*. IEEE, 2018, pp. 189–197.
- [24] S. Havermans, L. Baumgaertner, J. Roberts, M. Wallum, and J. Caballero, "Fuzzing space communication protocols," in *Proceedings of the 3rd Workshop on the Security of Space and Satellite Systems (SpaceSec '25)*. San Diego, CA, USA: Internet Society, Feb. 2025. [Online]. Available: <https://dx.doi.org/10.14722/spacesec.2025.23012>